

Part 1:

```
● jooke@MacBook-Air--jooke CSS 314 % python3 part1_phantom_bug.py
Run 1: hits = 30336897, pi = 2.42695176
Run 2: hits = 28684658, pi = 2.29477264
Run 3: hits = 29752650, pi = 2.380212
Run 4: hits = 29859242, pi = 2.38873936
Run 5: hits = 28477355, pi = 2.2781884
```

Run	Hits	Pi
1	30,336,897	2.42695176
2	28,684,658	2.29477264
3	29,752,650	2.38021200
4	29,859,242	2.38873936
5	28,477,355	2.27818840

Observation:

The result is not close to the real value of pi. The values are between 2.28 and 2.43. This happens because several threads update the same shared variable at the same time, so some updates are lost.

Part 2

```
● jooke@MacBook-Air--jooke CSS 314 % python3 part2_synchronization.py
Pi = 3.142124
Time: 1218.5297910000002 ms
```

Version	Runtime
Single-thread	1218.53 ms
4 threads + Lock	your measured synchronized time

Part 3

```

1 threads: 13809.71 ms, pi = 3.141694
2 threads: 13833.67 ms, pi = 3.141489
4 threads: 14169.61 ms, pi = 3.141594
8 threads: 14112.66 ms, pi = 3.141816
16 threads: 14130.49 ms, pi = 3.141679
32 threads: 14652.38 ms, pi = 3.141654

```

RESULTS

```

-----
1 threads | 13809.71 ms | 1.00x | 100.0%
2 threads | 13833.67 ms | 1.00x | 49.9%
4 threads | 14169.61 ms | 0.97x | 24.4%
8 threads | 14112.66 ms | 0.98x | 12.2%
16 threads | 14130.49 ms | 0.98x | 6.1%
32 threads | 14652.38 ms | 0.94x | 2.9%
ionke@MacBook-Air--ionke CSS 314 % ▮

```

Threads (T)	Runtime (ms)	Speedup vs. 1 Thread	Efficiency
1	13809.71	1.00x	100.0%
2	13833.67	1.00x	49.9%
4	14169.61	0.97x	24.4%
8	14112.66	0.98x	12.2%

Question 1

Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?

I think the main reason is that my CPU has 8 cores, so 16 threads cannot run on 16 physical cores at the same time. Also, Python threads have GIL overhead. Because of this, adding more threads does not make the CPU twice as fast. In my results, 16 threads took 14130.49 ms, while 8 threads took 14112.66 ms, so there was almost no improvement.

Question 2

Why was the synchronized version in Part 2 slower than running on one single core?

The synchronized version was slower because all threads had to wait for the lock before changing the shared counter. This creates extra synchronization overhead. In Python, the GIL also limits CPU-bound threads. So, using multiple threads for this calculation can be slower than using one thread.